Selection Sort

Ordenamiento por selección: explicación detallada del algoritmo y de su visualización con bolos en Graph Maker

1. ¿Qué es y qué hace?

Selection Sort (ordenamiento por selección) es uno de los algoritmos de ordenamiento más simples. Su idea es muy directa: en cada vuelta **busca el elemento más pequeño** de la parte que todavía no está ordenada y lo coloca en su posición definitiva, al inicio de esa parte. Después repite lo mismo con el resto.

Se llama "por selección" porque en cada ronda *selecciona* el mínimo. Una característica importante es que cada elemento, una vez colocado, **ya no se vuelve a mover**: queda en su lugar final.

Analogía

Imagina que tienes varias cartas sobre la mesa y quieres ordenarlas de menor a mayor. Revisas todas, tomas la más pequeña y la pones primera a tu izquierda. Luego revisas las que quedan, tomas la más pequeña y la pones segunda, y así sucesivamente hasta que no queden cartas por revisar.

2. Funcionamiento paso a paso

- El arreglo se divide en dos zonas: la **zona ordenada** (al inicio, comienza vacía) y la **zona sin ordenar** (el resto).
- Para la posición **i** (empezando en 0) se asume que el mínimo está en i.
- Se recorre con **j** desde i + 1 hasta el final. Cada vez que se encuentra un valor menor que el mínimo actual, se actualiza la posición del mínimo.
- Al terminar el recorrido, si el mínimo no estaba ya en i, se **intercambia** con el elemento de la posición i.
- La posición i pasa a formar parte de la zona ordenada. Se avanza i en 1.
- Se repite hasta que i llegue a n - 2. El último elemento queda solo y ya es el mayor, por lo que no hace falta procesarlo.

3. Ejemplo resuelto

Se ordena el arreglo **[5, 3, 8, 1, 4]** (n = 5):

Vuelta	Qué ocurre	Arreglo resultante
Inicio	Arreglo original.	[5, 3, 8, 1, 4]

Vuelta	Qué ocurre	Arreglo resultante
i = 0	Se busca el menor entre las posiciones 0 a 4: es el 1 (posición 3). Se intercambia con el 5.	[**1**, 3, 8, 5, 4]
i = 1	El menor entre las posiciones 1 a 4 es el 3, que ya está en su lugar. No hay intercambio.	[**1, 3**, 8, 5, 4]
i = 2	El menor entre las posiciones 2 a 4 es el 4 (posición 4). Se intercambia con el 8.	[**1, 3, 4**, 5, 8]
i = 3	El menor entre las posiciones 3 y 4 es el 5, que ya está en su lugar. No hay intercambio.	[**1, 3, 4, 5**, 8]
Fin	El 8 quedó solo al final y ya está en su lugar.	[1, 3, 4, 5, 8]

Observa: aunque el arreglo quedó ordenado desde la vuelta i = 2, el algoritmo igual sigue haciendo todas las comparaciones de las vueltas restantes. Selection Sort no se da cuenta de que ya terminó; esa es una de sus debilidades.

Cómo se visualiza en la aplicación

En la pantalla cada número del arreglo se dibuja como un **bolo de boliche**. El tamaño del bolo es proporcional a su valor: el bolo más pequeño representa el valor mínimo y el más grande el valor máximo. El número aparece dentro del bolo. Se pueden usar entre 1 y 20 bolos, con valores entre 1 y 99.

Los controles de la pantalla son los mismos en los cuatro algoritmos:

- **Slider (barra superior):** elige la cantidad de bolos (1 a 20). Genera la secuencia 1, 2, 3, ... n. Se deshabilita mientras se ordena.
- **Cronómetro:** muestra el tiempo en formato mm:ss.d. Se actualiza cada 100 ms con un Timer y se detiene cuando termina el ordenamiento.
- **Mezclar:** desordena los valores actuales con shuffle().
- **Aleatorio:** genera valores aleatorios entre 1 y 99 (pueden repetirse).
- **Ingresar:** permite escribir los valores a mano, separados por coma o espacio, con validación (solo números, de 1 a 99, máximo 20 valores).
- **Guardar / Cargar:** guarda y recupera listas de valores. El grupo de guardado es compartido, así que lo que se guarda en un algoritmo se puede cargar en los otros.
- **Resolver:** inicia el algoritmo con animación.
- **Velocidad:** el desplegable multiplica las pausas de la animación (y la duración con la que los bolos suben y bajan). Se puede cambiar incluso mientras se ordena.

Variables de estado que usa la animación

- **_i:** la posición que se está fijando en la vuelta actual.
- **_j:** el bolo que se está comparando en este momento.

- **_min:** la posición del menor valor encontrado hasta ahora.
- **_fijos:** cuántos bolos del inicio ya quedaron en su lugar definitivo.
- **_ordenando:** indica si el algoritmo está corriendo (deshabilita los botones).

Colores en la pantalla Selection Sort

Color del bolo	Significado en la pantalla
Naranja	El menor valor encontrado hasta el momento (_min).
Amarillo	El bolo que se está comparando contra el mínimo (_j).
Azul claro	La posición que se está fijando en esta vuelta (_i).
Verde claro	Bolos que ya están en su posición final (zona ordenada).
Blanco	Bolos todavía sin procesar.

Los bolos **_j** y **_min** se elevan ligeramente para destacar la comparación. Tras cada comparación hay una pausa de 120 ms y tras cada intercambio una pausa de 350 ms, ambas multiplicadas por el factor de la velocidad elegida.

4. Código clave

Este es el núcleo del algoritmo tal como está en la app (sin las llamadas de animación):

```
for (int i = 0; i < n - 1; i++) {
  int indiceMin = i;

  for (int j = i + 1; j < n; j++) {
    if (_valores[j] < _valores[indiceMin]) {
      indiceMin = j;            // nuevo menor encontrado
    }
  }

  if (indiceMin != i) {         // intercambio
    final temp = _valores[i];
    _valores[i] = _valores[indiceMin];
    _valores[indiceMin] = temp;
  }
}
```

5. Complejidad

Característica	Valor	Explicación
Mejor caso	$O(n^2)$	Aunque el arreglo ya esté ordenado, siempre recorre toda la zona sin ordenar para buscar el mínimo.
Caso promedio	$O(n^2)$	Doble ciclo anidado.
Peor caso	$O(n^2)$	Mismo número de comparaciones que en los otros casos.

Característica	Valor	Explicación
Comparaciones	n(n - 1) / 2	Siempre son exactamente las mismas, sin importar el orden inicial.
Intercambios	máx. n - 1	Es su mayor virtud: hace muy pocos intercambios.
Memoria extra	O(1)	Ordena en el mismo arreglo (in-place).
¿Estable?	No	Puede cambiar el orden relativo de elementos iguales. Ejemplo: [4a, 4b, 1] -> al intercambiar 4a con 1 queda [1, 4b, 4a].

6. Ventajas y desventajas

Ventajas

- Es muy fácil de entender e implementar.
- Hace muy pocos intercambios (máximo n - 1), útil si escribir en memoria es costoso.
- No necesita memoria adicional.
- Su tiempo es predecible: no depende del orden inicial de los datos.

Desventajas

- Es lento con muchos datos: $O(n^2)$ siempre, incluso con datos casi ordenados.
- No es estable.
- No aprovecha que los datos ya estén parcialmente ordenados.

7. ¿Cuándo usarlo?

Es adecuado para listas pequeñas, para fines didácticos y cuando el costo de intercambiar elementos es mucho mayor que el de compararlos. Para listas grandes conviene usar algoritmos como Merge Sort.